Acknowledgement

This document referenced work from A/P Nicholas Vun and Dr Loke Yuan Ren.

Overview

This document contains information related to image recognition task for the MDP. Divided into three sections: **OpenCV setup, Multi-Threading on RPI and Neural Network Training for Image Recognition**. The OpenCV and Multithreading topic is discussed in more detail in separate documents created by A/P Vun and Dr Loke.

OpenCV

OpenCV is an open-source package containing many libraries for various purpose. Some of the features available are listed below.

- Read and write images.
- Capture and save videos.
- Process images (filter, transform).
- Perform feature detection.
- Detect specific objects such as faces, eyes, cars, in the videos or images.
- Analyze the video, i.e., estimate the motion in it, subtract the background, and track objects in it.

Numerous tutorials can be found online. Below are two useful URL to explore. The Pyimage site contains information beyond OpenCV.

OpenCV site: https://docs.opencv.org/4.x/d9/df8/tutorial_root.html
Pyimage: https://pyimagesearch.com/

Setup instructions for OpenCV can be found in the NTULearn under Content -> Technical Materials -> Raspberry Pi folder.

The libraries in this package can be used for simple image processing (enhancing, resizing, cropping etc) in combination with the NN image recognition models. Or it could also be used by itself for image recognition purpose.

Note that you are not restricted in which approach/package you want to use for the image recognition task. OpenCV is a recommendation, not mandatory.

Multi-threading in RPI

With increased complexity in many embedded applications, operating system (OS) has become an essential element in many modern embedded systems. Multitasking is probably the main advantage for using an OS as it means multiple tasks can run 'concurrently' in a hosted environment i.e. one running with OS, multiple programs (tasks) can be launched separately with their executions managed by the OS using time multiplexing. However it is often better to have a single executable that can launch multiple tasks which allow the use of shared memory to enable simpler IPC (inter process communication), sharing of data/parameters.

The most common way to launch multiple tasks within a program is through multithreaded programming. Multithreading separates a process into many execution threads, each of which runs independently. Some benefits of multithreaded programming.

- Better responsiveness for the application.
- Improved program structure.
- Less system resources needed.
- Simpler IPC.
- More efficient use of multi-core processor.

Detail information on developing multi-threading on RPI can be found in the NTULearn Content -> Technical Materials -> Raspberry Pi folder. This includes a laboratory manual to walk student through developing a multithreaded application, including some considerations such as critical section protection, semaphores, and synchronizations.

Image recognition via Convolutional Neural Network (CNN) modelling

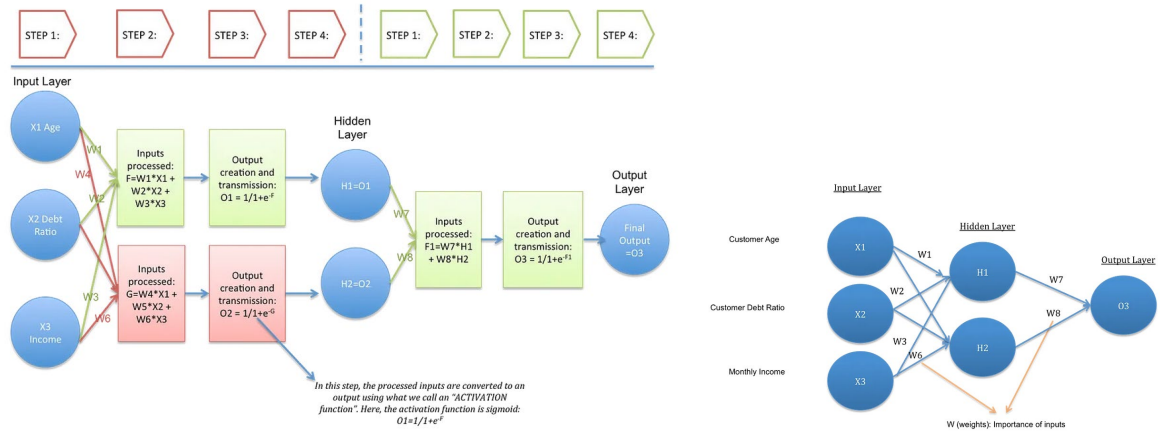
CNN is a popular approach to image recognition. There are many frameworks for neural network modelling in the market. We will only touch on the Tensorflow framework in this document. The quiz will also be based on the Tensorflow approach.

Students can choose to do the inference (during real-time image recognition) of the trained CNN model on the RPI or the PC. Doing it on the PC result in faster inferences but will need to incur the delay resulted from image transfer between RPI and PC. Doing the inference on RPI will reduce the image transfer delay but inferences will be slower as the processor of RPI is slower than that in the PC.

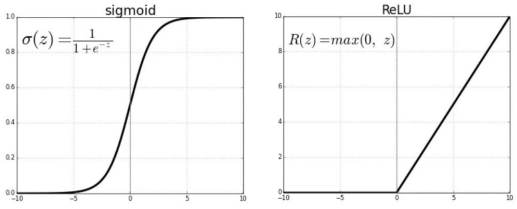
A. Deep Learning

Deep learning is based on a simplified idea of how the human brain might work. In deep learning, a network of simulated neurons, represented by arrays of numbers, is trained to model the relationships between various inputs and outputs. Different architectures, or arrangements of simulated neurons, are useful for different tasks. We will focus on the **Convolutional Neural Network (CNN)** architecture, which is popular for image recognition tasks. A typical deep learning process involves the following tasks: Goal setting, data collection, model design, model training, running inference to obtain predictions.

The 'model' mentioned above is a network of simulated neurons represented by arrays of numbers arranged in layers. These numbers are known as **weights** and **biases**, or collectively as the network's parameters. Figure below shows a model with three input neurons/nodes, a hidden layer with two nodes and an output layer with one node. Take node of how the **weights** and **activation function** are used to calculate to value of the hidden node output.



Sample Activation Functions

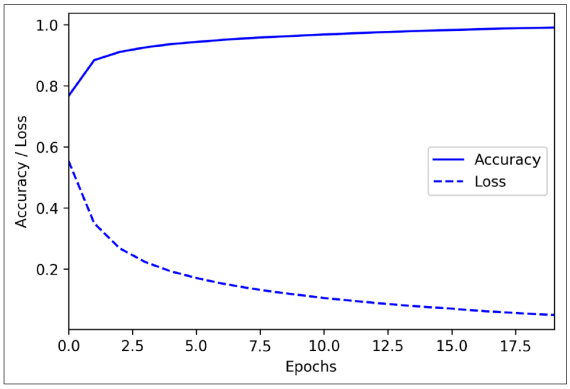


mage source: Towardsdatascience.com

Figure 1: DNN architecture

When you use a 'model' e.g. MNIST, you are using the derived set of weights and biases from the training done with the MNIST dataset.

When training a model for a particular task, the model's weights start out with random values, and biases typically start with a value of 0. During training, batches of data are fed into the model, and the model's output is compared with the desired output. An algorithm called **backpropagation** adjusts the **weights** and **biases** incrementally so that over time, the output of the model gets closer to matching the desired value. Training is measured in epochs and continues until we decide to stop. **One epoch is the number of times the training algorithm will go through the entire data set.** Each epoch consists of multiple **iterations**, each iteration involves passing one **batch** of data through the training algorithm to update the weights and bias**.** Iterations continue until the entire data set is used. We generally stop training when a model's performance stops improving, also known as **converged**. To determine whether a model has converged, we can analyze graphs of its performance during training. Two common performance metrics are **loss** and **accuracy**. The loss metric gives us a numerical estimate of how far the model is from producing the expected answers, and the accuracy metric tells us the percentage of the time that it chooses the correct prediction. A perfect model would have a loss of 0.0 and an accuracy of 100%, but real models are rarely perfect.



Source: TinyML. Pete Warden & Daniel Situnayake

Figure 2: DNN model accuracy and loss vs training epochs

To attempt to improve the model's performance, we can change our model architecture, and we can adjust various values used to set up the model and moderate the training process. These values are collectively known as **hyperparameters**, and they include variables such as the number of training epochs to run and the number of neurons in each layer. Each time we make a change, we can retrain the model, look at the metrics, and decide whether to optimize further.

But note that the model you are training may not converge or may not obtain the desired accuracy. The two most common reasons a model fails to converge are **underfitting** and **overfitting**. When a model is underfit, that means it has not yet been able to learn a strong enough representation of these patterns to be able to make good predictions. This can happen for a variety of reasons, most commonly that the architecture is too small to capture the complexity of the system it is supposed to model or that it has not been trained on enough data. When a model is overfit, that means it has learned its training data too well. The model performance very well with its training data, but it is not able to generalize its learning to data it has not previously seen. This often happens because the model has memorized every aspect of the training data too well, or it has learned to rely on a shortcut present in the training data but not in the real world. E.g. if the set of dog photos are all taken outdoor while the set of cat photos are all taken indoor, the model may use outdoor/indoor as a differentiating factor in deciding whether the animal is a cat or dog. Some means of mitigating overfitting is to reduce the complexity of the model, or to have a more varied dataset.

B. Tensorflow and Keras

TensorFlow is a set of tools for building, training, evaluating, and deploying machine learning models. Originally developed at Google, TensorFlow is now an open-source project built and maintained by thousands of contributors across the world. It is the most popular and widely used framework for machine learning. Most developers interact with TensorFlow via its Python library. Keras is TensorFlow's high-level API that makes it easy to build and train deep learning networks. TensorFlow Lite is a set of tools for deploying TensorFlow models to mobile and embedded devices. We will only cover the basics of Keras and Tensorflow in this doc.

Tensor

The 'tensor' here refers to a list that can contain either numbers or other tensors - similar to (but not exactly the same as) an array.

The structure of a tensor is known as its **shape**, and they come in multiple dimensions. We talk about tensors throughout this book, so here is some useful terminology:

- **Vector**
  A vector is a list of numbers, like an array. It's a tensor with a single dimension (a 1D tensor). The following is a vector of shape (5,) because it contains five numbers in a single dimension. E.g. **[42 35 8 643 7]**

- **Matrix**
  A matrix is a 2D tensor, like a 2D array. The following matrix is of shape (3, 3) because it contains three vectors of three numbers. E.g.

  [[1 2 3]
  [4 5 6]
  [7 8 9]]

- **Higher-dimensional tensors**
  Any shape with more than two dimensions is just referred to as a tensor. Here's a 3D tensor that has shape (2, 3, 3) because it contains two matrices of shape (3, 3). E.g.

  [[[10 20 30]
  [40 50 60]
  [70 80 90]]
  [[11 21 31]
  [41 51 61]
  [71 81 91]]]

- **Scalar**
  A single number, known as a scalar, is technically a zero-dimensional tensor. E.g. 42.

C. Sample Code

We will walk through a image classification sample code to have a feel of how to train a CNN for image recognition task. The actual code is in the link supplied. This document will elaborate on some key points. The codes are taken from tutorials in Google Tensorflow site,

Image Classification

URL: https://www.tensorflow.org/tutorials/images/classification

This tutorial shows how to classify images of flowers using a **tf.keras.Sequential** model and load data using **tf.keras.utils.image_dataset_from_directory**. It demonstrates the following concepts.

- Efficiently loading a dataset off disk.
- Identifying overfitting and applying techniques to mitigate it, including data augmentation and dropout.

It follows a basic machine learning workflow.

- Examine and understand data.
- Build an input pipeline.
- Build the model.
- Train the model.
- Test the model.
- Improve the model and repeat the process.

The tutorial also includes the steps required to convert a **Tensorflow** model to a **TensorflowLite** model. TensorflowLite model consume less memory and are more suitable for deployment on embedded systems. This doc will not elaborate on the conversion process, will leave it as an extension for students who wish to use TensorflowLite.

Dataset download and storage.

```
import pathlib
dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"
data_dir = tf.keras.utils.get_file('flower_photos', origin=dataset_url, untar=True)
data_dir = pathlib.Path(data_dir)
```

The dataset used in this tutorial is downloaded from google server. Do check how the images are stored and labelled in the file directories so you can adapt your captured images to this framework. Alternatively, there is a tutorial that show users how to customize data loading code from scratch.
URL: https://www.tensorflow.org/tutorials/load_data/images

The following screen shot shows how the images organized. Notice the resolutions are different for each image. More on this later.

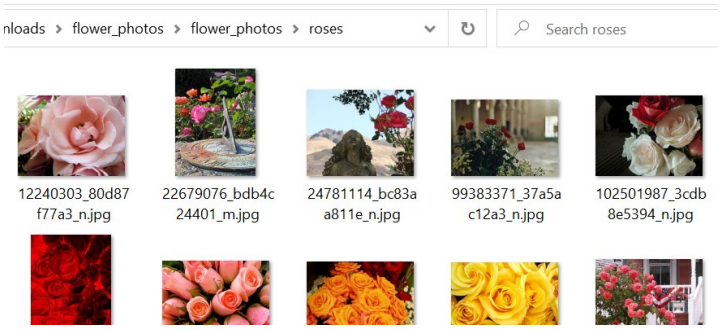


Figure 3: PC directory structure of downloaded images

User will need to create a dataset as the input data for model training, from the image data downloaded from Google. The corresponding code to use for Keras are as follows.

```
train_ds = tf.keras.utils.image_dataset_from_directory(
  data_dir,
  validation_split=0.2,
  subset="training",
  seed=123,
  image_size=(img_height, img_width),
  batch_size=batch_size)
```

```
batch_size = 32
img_height = 180
img_width = 180
```

Figure 4

In model training, there are typically three sets of data: **Training**, **Validation** and **Testing**. Training set is the data used for the model training, validation set is the data used for evaluating the accuracy of the model after one iteration, outcome of the evaluation will be used by the **backpropagation** algorithm to tweak the model parameters to improve its accuracy. Testing set is used after the model training completes to have test the model using an independent test data set. Typical ratio for **Training:Validation:Testing is 60:20:20**. Or if you are dealing only with training and validation data, the typical ratio is **80:20**.

The 'validation_split =0.2" implies 20% of the data will be partitioned for validation purpose. The image **height** and **width** are initialized as **180x180**, meaning the **image_dataset_from_directory** api will **resize** the original images to 180x180. You can view the original and the resized image in the dataset to see what has been done. The "**batch_size=32**" configures the training algorithm to select 32 images from the dataset for one training iteration, after which the respective weights and bias in the network are updated. This process will continue until the entire dataset is used. The number of epochs refers to the number of times the entire dataset is passed through the training algorithm. Performing the training in batches typically result in faster convergence because a batch of data samples fits the distribution of the entire dataset better than one single data sample.

The folder name of will be used as the **class_names** by Keras. This is used subsequently for image classification.

```
class_names = train_ds.class_names
print(class_names)
```

```
['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
```

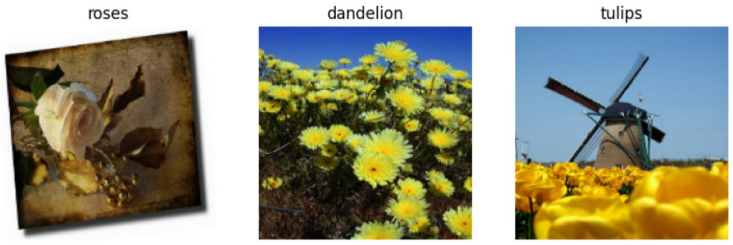


Figure 5

The labels_batch contains numerical label of the five classes of flowers. Note that the image_batch tensor has 32 images, 180x180 in resolution and 3 channels (RGB).

```
print(labels_batch)
for image_batch, labels_batch in train_ds:
  print(image_batch.shape)
  print(labels_batch.shape)
  break
```

```
tf.Tensor([2 1 3 1 4 1 2 3 4 3 3 0 4 4 3 1 0 2 0 4 1 1 1 2 3 1 4 0 3 4 1 1], shape=(32,), dtype=int32)
(32, 180, 180, 3)
(32,)
```

Figure 6

Data Normalization

The **scale** of data can be very different from each other and different from other parameters. Image data are typically one byte long (value 0-255). Dividing by 255 will bring the data set range to between **0 and 1**. This technique is known as normalization and is commonly used ensure that input data to the model have similar scale as each other. Two techniques are introduced in the tutorial, the **rescaling** layer approach will be discussed here, see “Basic Keras Model” section for details.

Basic Keras Model

The code used to construct the model is follows.

```
num_classes = len(class_names)

model = Sequential([
  layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
  layers.Conv2D(16, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(32, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(64, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Flatten(),
  layers.Dense(128, activation='relu'),
  layers.Dense(num_classes)
])
```

Figure 7

The **Rescaling layer**, as mentioned previously, will ensure that the input data will be between 0 and 1. A CNN model is used here. Check out appendix A for a short discussion on CNN basics. In the first **Conv2D** function, the first argument ‘16’ implies that 16 **filters** are applied onto the image separately to yield 16 **output channels**. The second argument ‘3’ means a 3x3 **kernel** (or mask) is applied to the image. **Stride length**, which is the number of pixels the kernel will move after each operation, has a

default value of 1. **Padding**='same' implies there is a padding of zero around the original image, this means the output image size will be the same as the input image (180x180). A **RELU** activation function is used for this layer. **Activation function** is used to transform the weighted inputs into an output value. See figures in section "A. Deep Learning" at the start of the document. The '**Flatten'** layer above converts the input tensor into a 1D tensor for computation. The '**Dense'** layer is a fully connected layer that connects every input to every node in that layer.

The model is then compiled. This is where some key configurations are specified. The "**adam optimizer**" is used here. **Optimizer** are used to mitigate the probability that the training will get stuck at **local minimum**, as opposed to a **global minimum**. A global minimum is desired as the prediction loss will be minimized. Another hyperparameter of interest is the learning rate, which controls size of the steps the algorithm takes in locating the global minimum. If the learning rate is large, we may skip the optimal solution. If it is too small, we may need many iterations to achieve convergence. To change the learning rate to 0.001, use "**optimizer=tf.keras.optimizers.Adam(learning_rate=0.001)**" instead of **optimizer='adam' in model.compile**. The default learning rate for adam optimizer in Keras is 0.0001. The loss function declared is used to measure the performance of each epoch of training, the loss is used by back propagation algorithms to adjust the weights and bias, and for user to determine when to stop the training.

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

Figure 8

The actual training of the model is executed by the following code. If things go well, the accuracy after each epoch should improve.

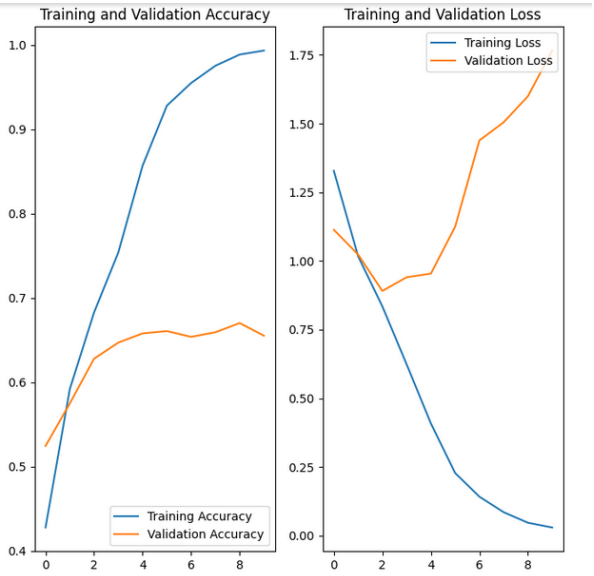
```
[ ] epochs=10
    history = model.fit(
      train_ds,
      validation_data=val_ds,
      epochs=epochs
    )
```

```
Epoch 1/10
92/92 [==============================] - 122s 1s/step - loss: 1.3280 - accuracy: 0.4278 - val_loss: 1.1134 - val_accuracy: 0.5245
Epoch 2/10
92/92 [==============================] - 118s 1s/step - loss: 1.0171 - accuracy: 0.5920 - val_loss: 1.0233 - val_accuracy: 0.5749
Epoch 3/10
```

Figure 9

Overfitting, Data Augmentation and Dropouts.

In the plots below, the training accuracy improves after each epoch but the validation accuracy stagnant at about 65%. This huge difference in the training vs validation accuracies is a sign of **overfitting**.



When there are a small number of training examples, the model sometimes learns from noises or unwanted details from training examples to an extent that it negatively impacts the performance of the model on new examples. This phenomenon is known as overfitting. It means that the model will have a difficult time generalizing on a new dataset.

There are multiple ways to fight overfitting in the training process. In this tutorial, **data augmentation** and **dropout** are used.

Overfitting generally occurs when there are a small number of training examples. Data augmentation takes the approach of generating additional training data from your existing examples by augmenting them using **random transformations** that yield believable-looking images. This helps expose the model to more aspects of the data and generalize better.

The code below generates more variations of the original data by applying various image transformation to the original image. The resultant augmented images are shown below as well.

```
data_augmentation = keras.Sequential(
  [
    layers.RandomFlip("horizontal",
                      input_shape=(img_height,
                                   img_width,
                                   3)),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
  ]
)
```

Figure 11

Another technique to reduce overfitting is to introduce **dropout regularization** to the network. When dropout is applied to a layer, it randomly drops out (by setting the activation to zero) a few output units from the layer during the training process. Dropout takes a fractional number as its input value, in the form such as 0.1, 0.2, 0.4, etc. This means dropping out 10%, 20% or 40% of the output units randomly from the applied layer.

A new model is created with data augmentation and dropout. Code shown below.

```
model = Sequential([
  data_augmentation,
  layers.Rescaling(1./255),
  layers.Conv2D(16, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(32, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(64, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Dropout(0.2),
  layers.Flatten(),
  layers.Dense(128, activation='relu'),
  layers.Dense(num_classes, name="outputs")
])
```

Figure 12

The number of **epochs** is increased slightly to 15 to ensure **convergence**.

```
epochs = 15
history = model.fit(
  train_ds,
  validation_data=val_ds,
  epochs=epochs
)
```

Figure 13

Validation accuracy is more aligned to **Training Accuracy**. The accuracy also improved to above 70%.

Figure 14

The last piece of code performs the real time prediction based on the trained model. In this example, a random **Sunflower** image was downloaded from another image source server for testing. The **PIL** (Python Image Library) image format is converted to a **Numpy** array and fed into the **model.predict** function for prediction. The **softmax** function normalize the values in the prediction array to between 0 and 1, essentially representing the **probability** of the test image belonging to each of the flower classes.

```
sunflower_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/592px-Red_sunflower.jpg"
sunflower_path = tf.keras.utils.get_file('Red_sunflower', origin=sunflower_url)

img = tf.keras.utils.load_img(
    sunflower_path, target_size=(img_height, img_width)
)
img_array = tf.keras.utils.img_to_array(img)
img_array = tf.expand_dims(img_array, 0) # Create a batch

predictions = model.predict(img_array)
score = tf.nn.softmax(predictions[0])

print(
    "This image most likely belongs to {} with a {:.2f} percent confidence."
    .format(class_names[np.argmax(score)], 100 * np.max(score))
)
```

Figure 15

Other examples

Some useful examples are listed below.

Load and preprocess images
https://www.tensorflow.org/tutorials/load_data/images

Transfer learning and fine tuning
https://www.tensorflow.org/tutorials/images/transfer_learning

Overfit and underfit
https://www.tensorflow.org/tutorials/keras/overfit_and_underfit

Tensorflow Tutorials main page
https://www.tensorflow.org/tutorials

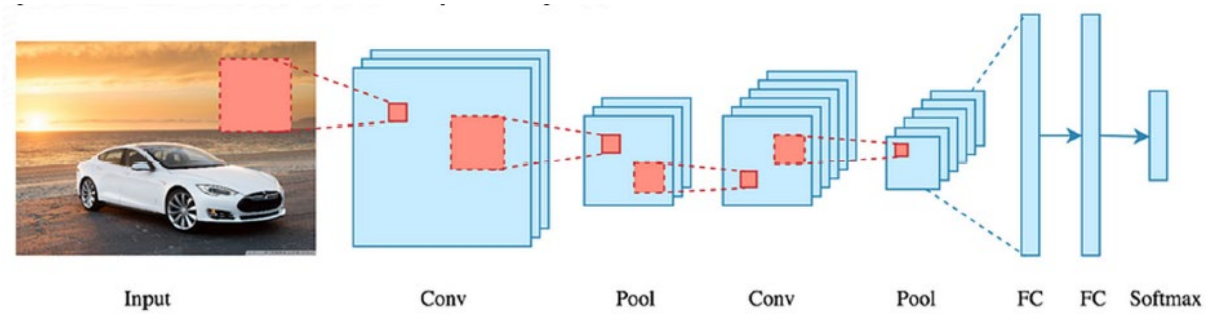
Appendix A - CNN

Overview

Convolutional neural network (**CNN**) is a class of artificial neural network that is popular in image analysis. CNNs use a mathematical operation called **convolution** in place of general matrix multiplication in at least one of their layers. They are specifically designed to process pixel data and are used in **image recognition and processing**. The convolution process is adapted from the process of applying a filter mask in conventional image processing. In CNN, the filter mask is known as the '**Kernel'**.

The kernels are used to **extract features** from images that is used by the NN model for designated tasks. As the model training progress, the algorithm automatically detects important features of the images from the dataset. CNN is computationally more efficient than the **fully connected network** found in a Deep Neural Network (**DNN**).

A typical CNN consist of series of convolution and pooling operation (layers). Output layer is typically a fully connected dense layer. One example below.

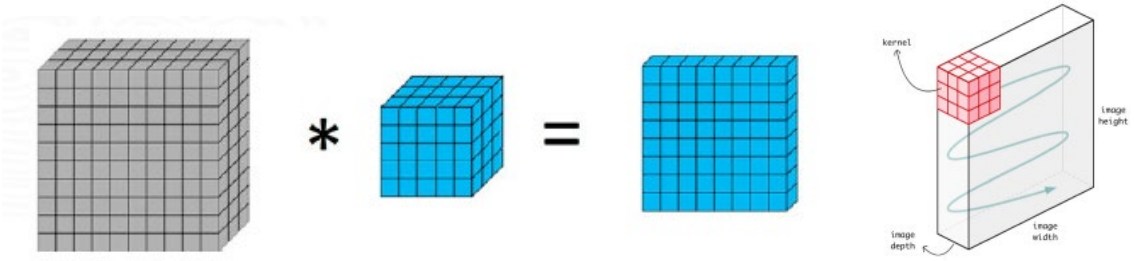


Main building block is the convolutional layer, which consists of kernels (filters) applied to its input tensor to detect and produce features maps. Real world image is represented by a 3D matrix.

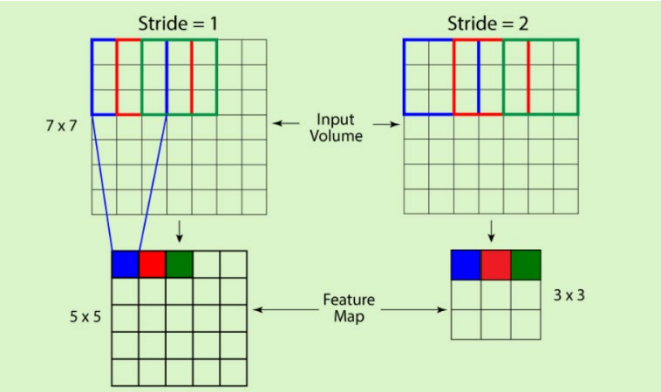
- **Height** and **width** correspond to the image size.
- **Depth** corresponds to the 3 colours (RGB).

Convolution Operation

The convolution operation is involves sliding a **3D kernel** with specific height, width and depth through an image or data. The kernel slides across the input tensor to perform the cross-correlation dot-product and weighted summed calculation. In a 2D convolution operation, the kernel slides along a two-dimensional path across the input tensor.

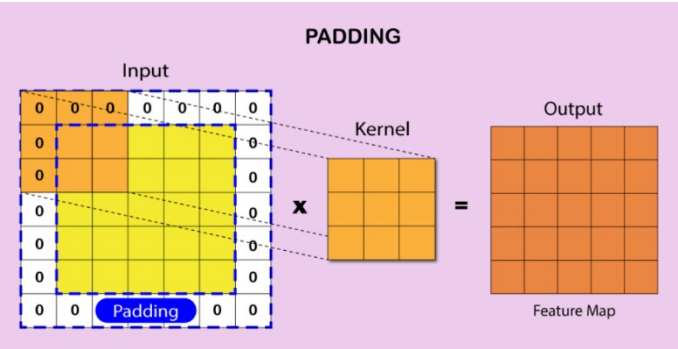


Convolution Operation – Stride



Stride refers to the number of pixels the kernel is moved at each step (after applying the kernel to one image pixel). Example of strides on the left. Notice the effect of stride length on the output image size.

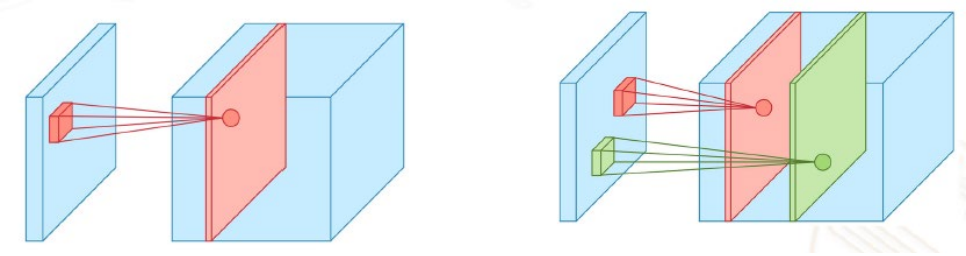
Convolution Operation – Padding



Stride within the input tensor reduces the size of the output tensor feature map shrinks as it progresses through the network, which is not desirable. Therefore, **padding** is used to retain the dimension and preserve the feature maps. This could be done by surrounding the input tensor with zeros or the values on the edge. See below.

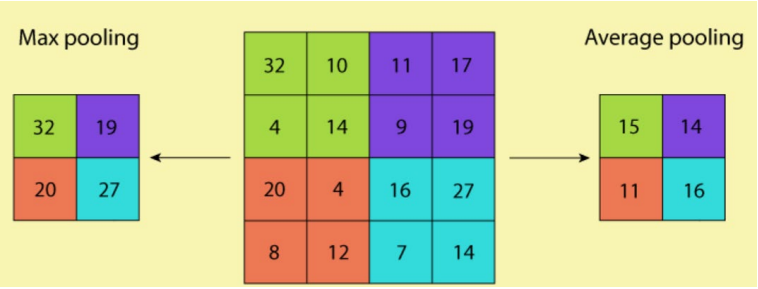
Multiple Kernel (Filters)

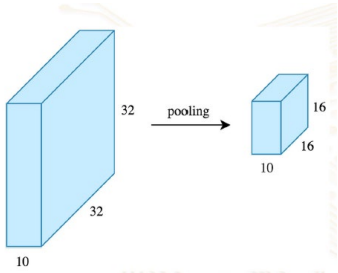
In practice, **multiple kernels** are applied to the input tensor and each kernel is used to detect certain **independent feature** of the input. Since each kernel produces one **feature map**, multiple kernels produce multiple maps that are stacked together, which means that depth of the output map increases as more kernels are used.



Pooling Operation

After each convolution, the output tensor is applied to a non-linear function. E.g. **ReLU** activation function. **Pooling** is then used to **reduce** the dimension of the output tensor. This reduces the parameters and shortens the training time but still retains the important feature information. Two popular pooling operations the max and average pooling. **Max pooling** takes the maximum value in the pooling window while **Average pooling** computes the average of the values within the pooling window. Below example uses a 2x2 pooling window and **stride length of 2**. This reduces the width and height by 50% and number of **parameters** (weights) will **decrease by 4 times**. A 2D window will not affect the depth though.

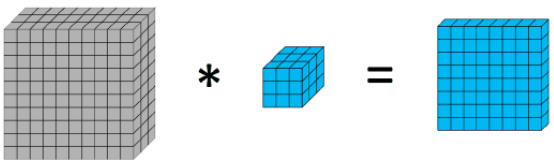




Depthwise Separable Convolution

There is another variation of convolution which is a **depthwise separable convolution**. The difference between this and convention CNN is illustrated below. Depthwise convolution treats the 3-input channel as independent channels and apply a 2D kernel to each channel, this result in **output depth of 3**. Conventional CNN, on the other hand, uses a 3D kernel and treat the 3 channels as one. This result in an **output with depth of 1**. **Conv2D** function in **Keras** implements conventional CNN. To use depthwise separable convolution, **DepthwiseConv2D** function needs to be used.

Conventional CNN.



Depthwise separable convolution.